Facultad de Ciencias de la Administración

TECNICATURA UNIVERSITARIA EN DESARROLLO WEB



Desarrollo para dispositivos móviles

Unidad 2 - Control de versiones & GIT

Qué es control de versiones?



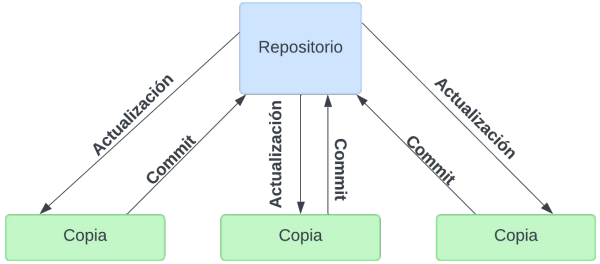
Cuando trabajamos en equipo, puede pasar que hagamos cambios en lugares similares y que conlleve a errores al momento de unir esos cambios.

El control de versiones nos ayuda a solucionar estos problemas y además provee:

- Un historial de los cambios que sufrió el archivo y nos permite volver a una versión anterior en caso de tener problemas.
- Trabajar en flujos de trabajo independientes, que pueden ser unidos entre sí verificando que no haya errores al integrarlos.
- La habilidad de trackear cada cambio con un msj describiendo que se hizo en ese cambio

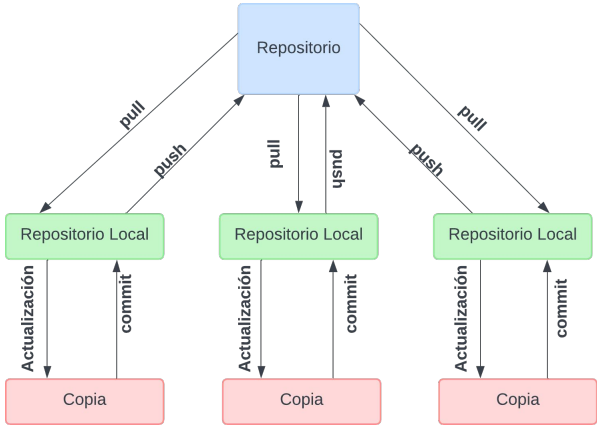


Centralizados



- Existe una única copia del código en un servidor y se commitean los cambios a ese servidor.
- En la máquina local solo se descarga la porción de código que queremos actualizar.

Descentralizados



- No solo se encuentra la copia en el servidor, sino que también en cada computadora donde se necesite.
- Se clonan los archivos y el historial de los mismos.

Qué es GIT?



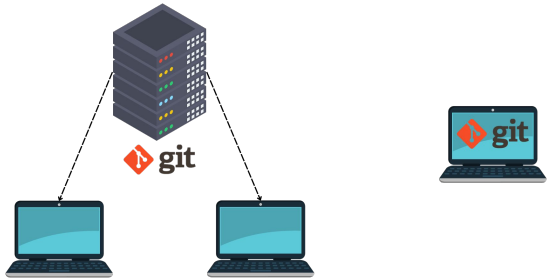
git

Es una herramienta open source para el control de versiones distribuida, diseñada para manejar proyectos de cualquier tamaño. Fue creada por Linus Torvalds, el creador de Linux, y hasta el dia de la fecha es la herramienta de versión de control más utilizada.



- Es gratis y de código abierto.
- Puede ser instalado en cualquier sistema operativo.
- Es descentralizado, no se depende de un solo servidor. Y cada contribuyente tiene una copia completa del código.
- Se puede usar en un servidor o como un cliente en una computadora.
- Las operaciones son rápidas gracias a que se guarda una copia local del repositorio.

- Permite llevar un historial de los cambios de repositorios privados hosteados por uno mismo o la empresa donde trabajamos, o publicar el código en servidores/hostings públicos tales como **Github**, **Gitlab** o **Bitbucket**.



Hay dos maneras de usar el cliente de git en nuestras computadoras:

- A través de una interfaz gráfica tales como Github Desktop, Sourcetree.
- Usando extensiones para los IDEs.
- Línea de comandos





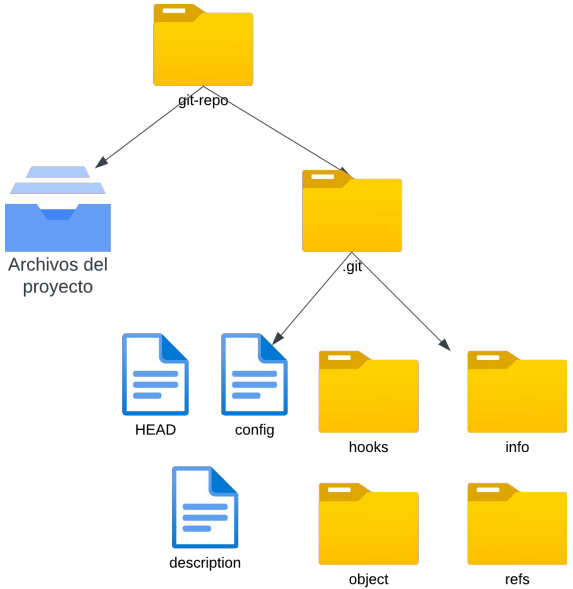
1. Descargar Git desde la página oficial https://git-scm.com/
2. Seguir los pasos del instalador.
3. Configurar el email y el nombre que se va a mostrar en cada commit.
   a. `git config --global user.email "email@example.com"`
   b. `git config --global user.name "username"`

Git - Inicializar un proyecto de git

Para inicializar un repositorio de git solo necesitamos ejecutar **`git init`** dentro de la carpeta del proyecto. Acepta diferentes variables, en este caso vamos a inicializar el proyecto con una rama por defecto.

```
//nombre_rama puede ser cualquiera, pero se recomienda que la inicial sea "main"
git init --initial-branch=nombre_rama
```

Este comando crea una carpeta .git donde se va a guardar toda la información del repositorio



- Hooks: Son scripts que son ejecutados al correr un comando de git
- Info: Se guarda información sobre el proyecto
- Objects: Donde se guarda el código en forma de snapshots (versionado).
- Refs: se almacenan las referencias a los commits, se encarga de hacer la asociación entre el hash del commit y un string “user-friendly”.

O también conocido como working directory. Representa el estado actual/versión del proyecto.

Comprende todos los archivos trackeados por GIT, así como también los nuevos que no han sido agregados a la lista de archivos trackeados aún.

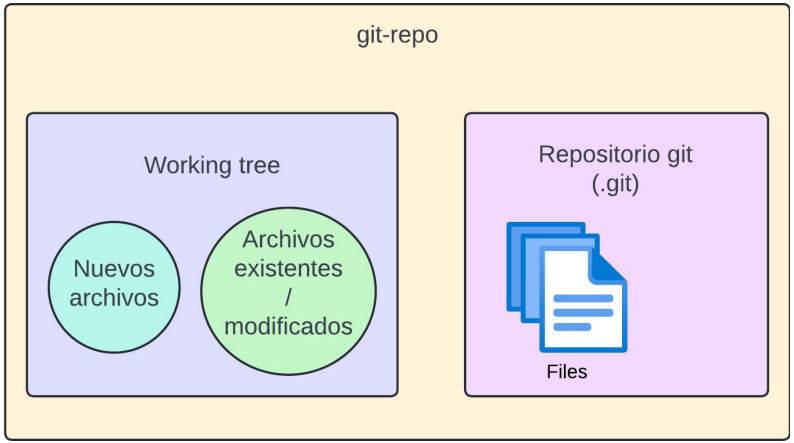
```
git status
```

```
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .idea/
        package.json

nothing added to commit but untracked files present (use "git add" to track)
```



Creamos un archivo con contenido para que nuestro working tree no esté vacío. Sin embargo para que el archivo sea trackeado por GIT tenemos que agregando al "**área staging**".

```
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        contenido.js

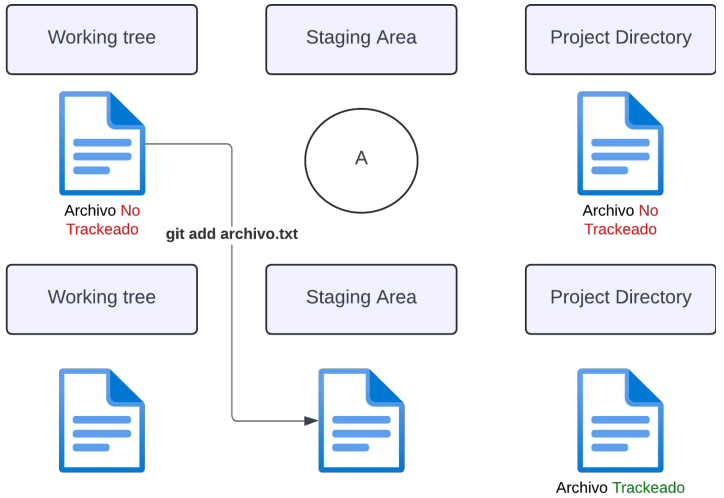
nothing added to commit but untracked files present (use "git add" to track)
```

Este área es un lugar intermedio entre el repositorio GIT y el working tree. Contiene la información de los archivos nuevos o modificados que van a ser incluídos en el próximo commit.

```
//archivo.ext es el path al archivo con el archivo incluido.
git add <archivo.ext>

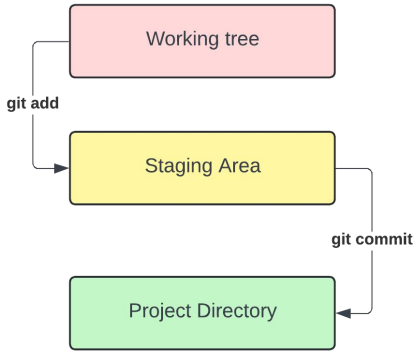
//git add -A para agregar todos los archivos
```

```
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   contenido.js
```



Un **commit** es la operación que guarda los cambios hechos al repositorio GIT. Cuando se “commitea” algo, se está creando una foto (**snapshot**) del proyecto.

```
git commit -m "Descripción detallada pero breve de lo que se hizo"
```



Luego de ejecutar el commit. Ejecutamos `git status`.

```
On branch main
nothing to commit, working tree clean
```

Por qué son importantes?

Ayudan a entender cuales son los cambios sufrió el proyecto.

Un mensaje bien definido simplifica el trabajo a los colaboradores para entender el **propósito** y **contexto** del cambio realizado en el proyecto

Un mensaje claro conlleva a un historial de commits que facilita la colaboración, debugging y el proceso de identificar qué cambios se hicieron en un momento específico

Buenas prácticas para un mensaje

- Claridad y ser conciso.
  Debe ser conciso, pero a la vez claro con lo que se hizo en ese commit, todo en una sola línea



- Usar título y contenido. **Solo si es necesario**
  En caso de necesitar proveer más información podemos usar un título y un contenido del mensaje. Para ellos escribimos `git commit` y nos abrirá un editor de texto como **nano** o **vim** (depende que tengamos instalado) para editar el mensaje.

```
Titulo:"Solucionado el error al crear la cuenta"
Contenido:"El error ocurria cuando el usuario trataba de crear la cuenta y tenia
seleccionado el rol profesor y ademas no mostraba el dropdown de la clase a la
cual iba a acuidar"
```

- Escribir en imperativo.

```
"Arreglé un error en el formulario de creación de cuenta"
"Se arregló un error en el formulario de creación de cuenta"
```

Git - Historial de commits

Para ver el historial de commits cronológicamente se tiene que ejecutar el comando `git log`

```
git log -n 2
git log --oneline
```

```
commit e5212c11f58e5ea8be9004f5b80f1884d602b2e5 (HEAD -> main)
Author: Franco Ruis Dias <franco.dias@datumsure.com>
Date:   Mon Sep 2 23:16:48 2024 -0300

    Nuevo archivo
```

- Commit hash, es el identificador único del commit.
- Author, es el autor del commit. Los valores los insertamos al comienzo de esta clase con git config
- Date, fecha y hora del commit
- El mensaje del commit.

Representa una línea independiente de desarrollo, permitiendo a los desarrolladores trabajar en nuevas funcionalidades,arreglar errores, experimentar con nuevas ideas en un ambiente aislado del resto del proyecto hasta que esté listo para la integración

Desarrollo en equipo o paralelo
Para optimizar el trabajo en equipo es muy común que se trabajen en varias cosas a la vez. Las ramas permites que cada uno trabaje en sus tareas sin interrumpir a otros compañeros.

Desarrollar una nueva tarea
Cada nueva tarea puede ser asignada a ser trabajada en una rama en particular, permitiendo a los desarrolladores enfocarse en esa tarea sin impactar la rama principal. Una vez que se termine se integra con la rama principal.

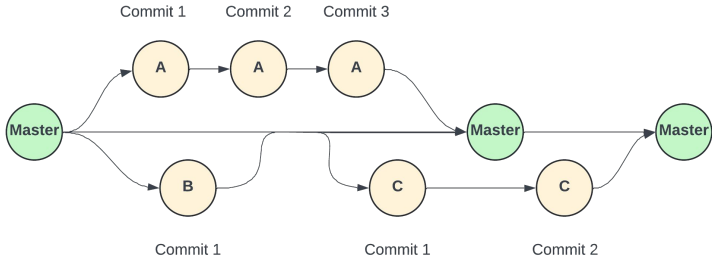
Arreglo de errores
Similar al punto anterior. Se aísla el problema para solucionarlo sin impactar la rama principal hasta que sea integrado.

Experimentación
Es muy normal querer probar cosas nuevas o mejoras al código. Entonces si funciona se integra, sino se descarta.

Administración de releases
En proyectos grandes se utilizan para manejar versiones. Se crea un rama estable que se usa para desplegar. Y si hay que hacer un hotfix el desarrollador lo hace directamente desde esa rama sin necesidad de arrastrar todos los cambios que el equipo está trabajando.

Para trabajar con las ramas usamos el comando `git branch` solo que después se usan variables diferentes.

```
//para listar
git branch
//Para crear. Reemplazar <nombre-rama> por un string
git branch <nombre-rama>
//Para borrar la rama.
git branch <nombre-rama> -D
```



Generalmente en el equipo se hablan de las convenciones para el nombre de ramas. Por ejemplo se puede hacer:

- `feat/<funcionalidad>`
- `fix/<funcionalidad-a-arreglar>`
- `hotfix/<funcionalidad-a-arreglar>`

Para cambiar entre ramas se usa `git checkout <nombre-rama>`. También se puede usar `git checkout -b <nombre-rama>` para crear la rama y moverse a esa rama creada.

Este proceso se lo conoce como merge o merging. Integra los cambios de una rama con la otra.

```
commit 89174eb9c519f864a6d30c57fd41bab3a7dbc404 (HEAD -> nueva-rama)
Author: Franco Ruis Dias <franco.dias@datumsure.com>
Date:   Tue Sep 3 19:02:59 2024 -0300

    feat: funcion para sumar

commit e5212c11f58e5ea8be9004f5b80f1884d602b2e5 (main)
Author: Franco Ruis Dias <franco.dias@datumsure.com>
Date:   Mon Sep 2 23:16:48 2024 -0300

    Nuevo archivo
```

```
Updating e5212c1..89174eb
Fast-forward
 contenido.js | 8 ++++++--
 1 file changed, 6 insertions(+), 2 deletions(-)
```

```
commit 89174eb9c519f864a6d30c57fd41bab3a7dbc404 (HEAD -> main, nueva-rama)
Author: Franco Ruis Dias <franco.dias@datumsure.com>
Date:   Tue Sep 3 19:02:59 2024 -0300

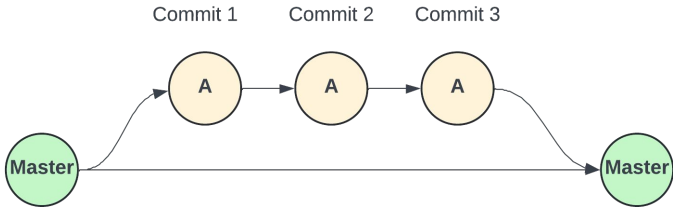
    feat: funcion para sumar

commit e5212c11f58e5ea8be9004f5b80f1884d602b2e5
```

Como se ve la nueva rama está un commit adelante de main. Para integrarlas, tenemos que **movernos** a la rama principal (main) donde queremos **integrar** nueva-rama, y desde main rama ejecutamos `git merge <nombre-rama>`

fast-forward merge

Este tipo de merge se ejecuta cuando la rama que queremos integrar (nueva-rama) tiene todos los commits de la rama principal (main), es decir, el histórico de los commits de estas ramas **no divergen**



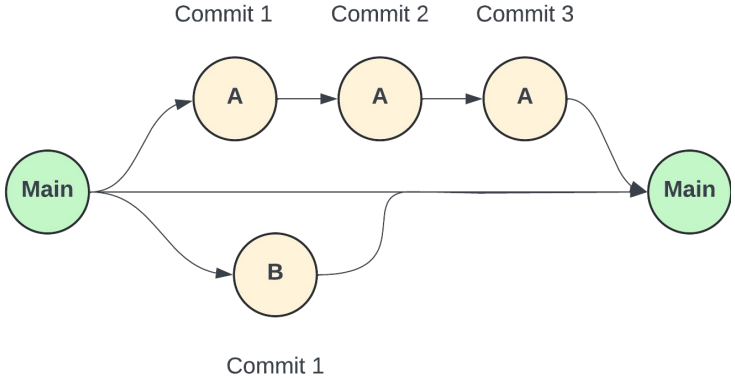
Three-way merge

Este merge se da cuando las ramas divergen. El término refiere a las 3 partes que están involucradas en el merge.

1. El ancestro más reciente que tienen en común (commit).
2. La versión del último commit de la rama que estamos posicionados (main).
3. La versión del último commit de la rama a integrar.

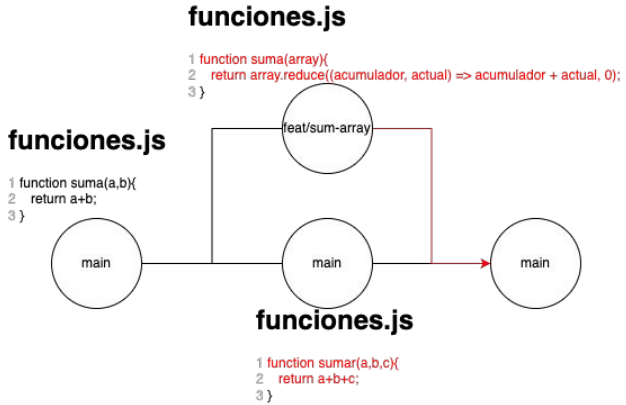
Funciona de la siguiente manera:

1. Git identifica los cambios hechos en cada rama desde de la **parte 1**.
2. Si cada rama tiene cambios en diferentes archivos o diferentes partes del mismo archivo. Git los va a **mergear automáticamente**.
3. Caso contrario **marcará los conflictos**.



También conocidos como **Merge Conflicts** suceden cuando generalmente cuando se hacen cambios en la mismas líneas de código en dos ramas diferentes y GIT no puede integrarlas automáticamente.

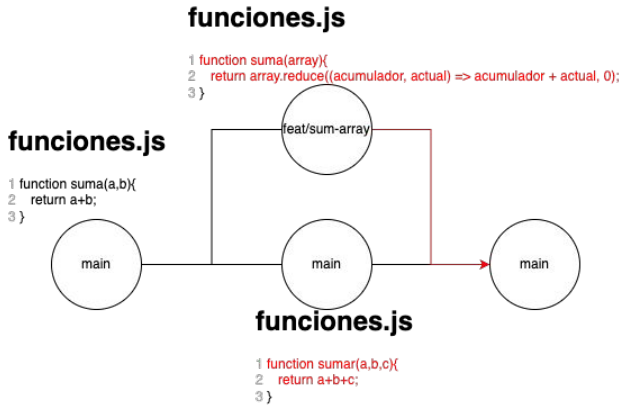
```
//Mensaje de error al querer mergear dos ramas divergentes
Auto-merging funciones.js
CONFLICT (content): Merge conflict in funciones.js
Automatic merge failed; fix conflicts and then commit the result.
```



Al querer integrar dos ramas GIT va a mostrar un error como el que está más arriba, mostrando los/el archivo/s con conflictos.

En cada archivo vamos a encontrar un patrón similar al siguiente:

```
<<<<<<< HEAD
function sumar(a, b,c) {
  return a + b + c;
=======
function suma(array) {
  return array.reduce((acumulador, actual) => acumulador + actual, 0);
>>>>>>> feat/sum-array
}
```



- `<<<<<<< HEAD` indica el comienzo de los cambios de la rama actual, en nuestro caso es main
- `=======` Separa los cambios de cada rama, arriba la rama actual mientras que abajo la rama que queremos integrar.
- `>>>>>>> feat/sum-array` indica el final de los cambios provenientes de la rama a integrar

Para solucionarlo solo basta con borrar el que no queremos y mantener el cambio deseado, eliminando las líneas agregadas por GIT.
Una vez modificado el archivo solo tenemos que ejecutar lo siguiente:

```
<<<<<<< HEAD
function sumar(a, b,c) {
  return a + b + c;
=======
function suma(array) {
  return array.reduce((acumulador, actual) => acumulador + actual, 0);
>>>>>>> feat/sum-array
}
```

```
git add <nombre-archivo>

git commit -m"fix: conflictos de integracion"
```

```
commit bd2843cb9b31b69676e3f9720457a4577ced3c0b (HEAD -> main)
Merge: 56fc21b 070a1a3
Author: Franco Ruis Dias <franco.ruisdias@uner.edu.ar>
Date:   Wed Sep 4 20:06:51 2024 -0300

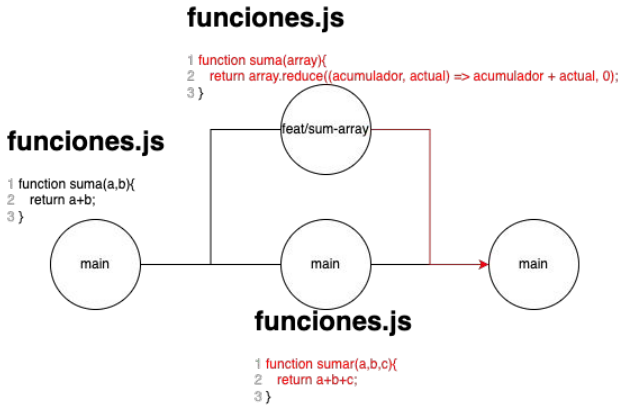
    fix: merge conflicts

commit 56fc21b926e21bc1501223cb3d8e9178355bc7f0
Author: Franco Ruis Dias <franco.ruisdias@uner.edu.ar>
Date:   Wed Sep 4 19:46:05 2024 -0300

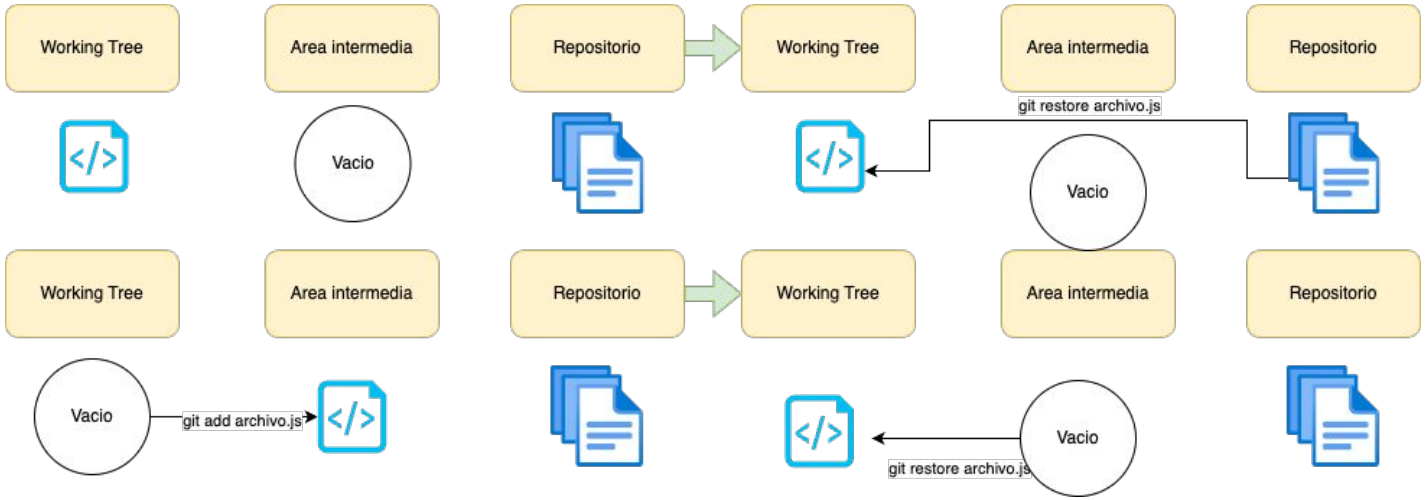
    feat: sumar 3 elementos

commit 070a1a3ab505781e678ddf2cf66f09ff30049bb8 (feat/sum-array)
Author: Franco Ruis Dias <franco.ruisdias@uner.edu.ar>
Date:   Wed Sep 4 19:45:35 2024 -0300

    feat: sumar items de un array
```



Hay momentos en los que necesitamos revertir los cambios antes de siquiera haberlos movido al **area intermedia** (**Stage**). Dependiendo donde se encuentre la versión anterior, se revierte a un archivo de **Stage** o del **Repositorio**.



Similarmente se pueden revertir los cambios que ya se encuentra en el área intermedia con el siguiente comando:

```
git restor --staged <archivo>
```

La bandera `--staged` le dice a GIT que tiene que restaurar cambios que se encuentran en el área intermedia. Después de ejecutar este comando notarán que los archivos se van a volver a encontrar en el **working tree**. Es decir no revierte los cambios del working tree, sino que los saca del área intermedia.



En el caso que se quiera revertirlos también, deberíamos agregar la bandera `- -worktree`

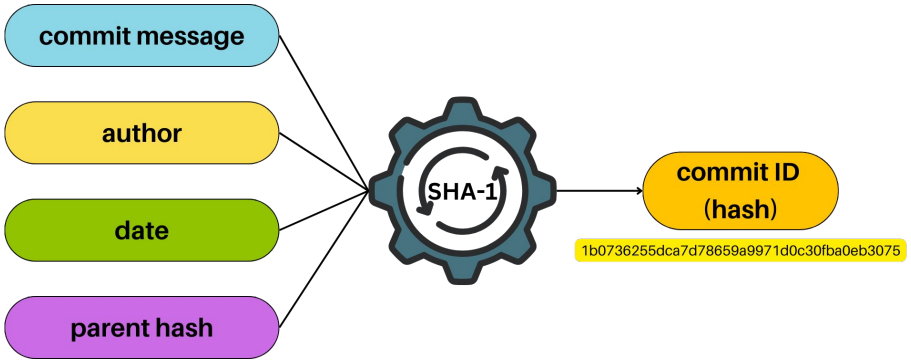
```
git restor --staged --worktree <archivo>
```

Hay momentos en los que necesitamos revertir los cambios a un commit en particular. En estos casos GIT crea un nuevo commit haciendo el reverso de los cambios realizados.

```
git revert HEAD

git revert <HASH_COMMIT>
```

- HEAD, es una referencia al último commit de la rama en la cual estamos parados
- HASH_COMMIT, es un **ID único** que nos ayuda a referenciar el commit



Existe un comando conocido como `git stash` que guarda los cambios temporalmente en un **stash** para que puedan ser vueltos a usar más adelante.
Guarda los cambios sin confirmar (sin commit) y los deshace para trabajar en un ambiente limpio de cambios.

Luego se puede volver a utilizar `git stash pop` para retomarlos.

Solo guarda cambios en los archivos, pero no archivos nuevos que hayan sido creados, es decir, no ejecutamos `git add`. Para ellos debemos agregar la bandera -u para que también guarde los archivos sin seguimiento